

Beginner Guide to Kubernetes Helm Charts

Writing individual YAML files for every Kubernetes deployment quickly becomes repetitive and difficult to manage across teams. Helm solves this by packaging your manifests into reusable templates that can be installed, updated, and rolled back with single commands.

What you'll learn:

1. Building Your First Helm Chart
2. Managing Environment Values
3. Upgrading and Reverting Deployments
4. Validating and Debugging Templates

Aman Raj Singh

Cloud Operations Engineer @ iManage

linkedin.com/in/mramanrajsingh

1

TIP 1 OF 4

1. Building Your First Helm Chart

Helm charts group your Deployments, Services, and Ingress into one manageable directory.

Running the create command produces a standard folder containing Chart.yaml, values.yaml, and a templates folder.

Chart.yaml stores metadata like chart version and app version.

The templates folder holds raw Kubernetes YAML files injected with Go template placeholders.

```
$# Scaffold a chart and verify directory layout
helm create my-web-app
tree my-web-app/
```

Cloud & DevOps Daily

Helm Charts Explained: Package Kubernetes Apps the Right Way

Monday, September 28, 2026

Swipe to tip 2 >> linkedin.com/in/mramanrajsingh

2

TIP 2 OF 4

2. Managing Environment Values

Instead of duplicating manifests for dev, staging, and prod, use values files to inject variables.

The default values.yaml holds fallback parameters like replica counts and port numbers.

You can pass distinct environment files during installation using the -f flag.

This makes your deployment portable without changing core template files.

```
$# Deploy using a specific environment configuration file
helm install web-prod ./my-web-app -f ./my-web-app/values-prod.yaml
```

Cloud & DevOps Daily

Helm Charts Explained: Package Kubernetes Apps the Right Way

Monday, September 28, 2026

Swipe to tip 3 >> linkedin.com/in/mramanrajsingh

3

TIP 3 OF 4

3. Upgrading and Reverting Deployments

Helm tracks release revisions every time you update your app.

Running helm upgrade applies configuration changes as a rolling update.

If a bad configuration or bug breaks the cluster, you can revert instantly.

The rollback command re-applies the exact manifests from an earlier revision number.

```
$# Upgrade the running app and roll back if needed
helm upgrade web-prod ./my-web-app --set image.tag=v2.1.0
helm history web-prod
helm rollback web-prod 1
```

Cloud & DevOps Daily

Helm Charts Explained: Package Kubernetes Apps the Right Way

Monday, September 28, 2026

Swipe to tip 4 >> linkedin.com/in/mrmanrajsingh

4

TIP 4 OF 4

4. Validating and Debugging Templates

A simple indentation typo in YAML can break an entire cluster deployment.

The `helm lint` command scans your chart for best practices and structural errors.

The `helm template` command renders the final YAML locally without contacting the Kubernetes API.

This helps you inspect generated manifests before pushing changes to git.

```
$# Test chart syntax and inspect output manifests
helm lint ./my-web-app
helm template test-release ./my-web-app --debug
```

Cloud & DevOps Daily

Helm Charts Explained: Package Kubernetes Apps the Right Way

Monday, September 28, 2026

See Key Takeaways on next page >> linkedin.com/in/mramanrajsingh

Key Takeaways

- + Always run 'helm lint' and 'helm template' before deploying to shared clusters.
- + Store production values in dedicated, version-controlled files (e.g., values-prod.yaml).
- + Use explicit image tags rather than 'latest' in values.yaml to ensure repeatable releases.
- + Check release history with 'helm history <release-name>' before performing rollbacks.
- + Keep Chart.yaml version (chart version) distinct from appVersion (application image tag).

Watch Out For

- ! Hardcoding environment variables inside templates/ directory files instead of values.yaml.
- ! Forgetting that 'helm uninstall' completely deletes the release and associated persistent state.
- ! Using incorrect YAML indentation inside values files, leading to silent variable parsing failures.

Follow for daily Cloud & DevOps guides!

linkedin.com/in/mrmanrajsingh